get_next_line Live Coding: Parameterized Separator

Prepared by Daniel Landi

The Professional Solution: Passing the Separator as a Parameter

The evaluation guideline explicitly states you must "support the use of a custom separator character, chosen by the reviewer" and "demonstrate that the modification works correctly using concrete examples." The most professional, engineer-level way to execute this in under 10 minutes is to pass the separator (`char sep`) down the pipeline.

Step 1: Update the Public Header

Modify the main prototype in `get_next_line.h` to accept the new delimiter parameter.

```c
// get_next_line.h
char *get_next_line(int fd, char sep);
```

Step 2: Update the Internal Pipeline (`get_next_line.c`)

Add `char sep` to your helper functions and replace the 5 hardcoded `'\n'` characters with the variable `sep`.

1. `clean_reserve` (1 change)

```c
char *clean_reserve(char *reserve, char sep)
{
    // ...
    while (reserve && reserve[i] && reserve[i] != sep)
        i++;
    // ...
}
```

2. `extract_line` (3 changes)

```c
char *extract_line(char *reserve, char sep)
{
    // ...
    while (reserve[i] && reserve[i] != sep)
        i++;
    // ...
    while (reserve[i] && reserve[i] != sep)
    {
        str[i] = reserve[i];
        i++;
    }
    if (reserve[i] == sep)
    {
        str[i] = reserve[i];
        i++;
    }
    // ...
}
```

3. `read_to_reserve` (1 change)

```c
char *read_to_reserve(int fd, char *reserve, char sep)
{
    // ...
    while (bucket && !ft_strchr(reserve, sep) && b > 0)
    // ...
}
```

Step 3: Wire it into the Main Function

Update `get_next_line` to accept the parameter and pass it down to your freshly updated helper functions.

```c
char *get_next_line(int fd, char sep)
{
    // ...
    reserve = read_to_reserve(fd, reserve, sep);
    if (!reserve)
        return (NULL);
    line = extract_line(reserve, sep);
    // ...
    reserve = clean_reserve(reserve, sep);
    return (line);
}
```

Step 4: Build the Concrete Example

To prove it works, write a quick `main.c` file that reads a text file separated by commas.

1. **Create a test file (`test.txt`):**

```
apple,banana,cherry,
```

2. **Write the testing framework:**

```c
#include "get_next_line.h"
#include <fcntl.h>
#include <stdio.h>

int main(void)
{
    int fd = open("test.txt", O_RDONLY);
    char *line;

    // Ask the reviewer: "What separator would you like to use? A comma?"
    while ((line = get_next_line(fd, ',')) != NULL)
    {
        printf("Extracted: %s\n", line);
        free(line);
    }
    close(fd);
    return (0);
}
```

Compile and run. When it cleanly prints out the fruits one by one, you will have completed the live coding section perfectly in about three minutes.